Distill I

Build and measure the evidence path

Data foundations, embeddings and retrieval Session 2 of 5 · 3 hours · source-aligned workshop

The evidence path is the product

- Preserve policy structure and tables.
- Retrieve the right version for the requested date.
- Measure expected clauses before generating answers.

Today’s workshop route

- 25 min data/retrieval concepts + 20 min walkthrough
- 90 min pair experiments, including a 10 min break
- 25 min retrieval gate review + 20 min recap

Showcase / a fee changes on 1 July

BEFORE THE CUTOFF

30 June sanction and lookup: base policy fee.

FROM THE CUTOFF

1 July sanction and lookup: linked circular fee.

SCOPE CHECK

A different product must not reuse those sources.

Run the dated-policy demonstration

```
python -m training.showcase --session 2
```

June: 0.35% / POL-HL-V3.

July: 0.30% / CIR-2026-07; quotes verified.

Data Foundations for LLMOps

Educative lesson 07

Concept → worked example → NexaAssist application

Documents carry meaning through structure

- Tables, headers and code blocks define useful boundaries.
- Extraction can mix unrelated columns and page furniture.
- Inspect representative parsed output before embedding.

Three forms of document noise

- Artifact noise: control characters and extraction debris.
- Layout noise: repeated page headers and split tables.
- Semantic noise: obsolete or contradictory policy text.

Embed-ETL has a safety order

Parse and normalize → inspect structure.

Redact sensitive fields → define stable chunk metadata.

Only then compute embeddings and publish searchable data.

Chunk around clauses, then size

```
def chunk_by_clause(elements: list[Element], max_characters: int, overlap: int) -> list[Chunk]:
    """Hybrid chunking: structure rules first, size rules second.

    Structure: one chunk per numbered clause ("6.2 Processing fee. ..."). A table or
    continuation line stays with the clause that introduces it, and a table is never split.
    Size: only a clause longer than `max_characters` is split. Its tables become their own
    chunks and its narrative is split at sentence boundaries with `overlap` characters
    repeated. Text before section 1 is section 0, "Front matter".
    """
    section_number, section = 0, "Front matter"
    clauses: list[_Clause] = [_Clause(section_number, section, None, None)]
    for element in elements:
```

Inspect how the clause-aware splitter handles oversized sections.

Practical 1 / observe real PDF chunks

```
Run: python -m training.session2.chunks
```

Compare maximum sizes 300, 1200 and 5000.

Expected counts: 40, 32 and 32; tables remain visible.

PAIR LAB / Predict → run → inspect → explain / uv run --no-sync

Small search units; readable context

Child: precise clause text with metadata.

Parent: the surrounding section used for generation.

Budgeting must preserve required amendment evidence.

Idempotency is an observable property

- Unchanged text and embedding schema → no re-embedding.
- Model or dimensions change → regenerate vectors.
- Stable clause identity avoids renumbering unrelated data.

Practical 2 / repeat ingestion offline

```
Run: python -m training.session2.ingest_twin
```

Predict embedded counts before each run.

Expected: 32 → 0 → 32 → 0.

PAIR LAB / Predict → run → inspect → explain / uv run --no-sync

PII controls belong before embedding

```
Run: python -m training.session2.redact
```

Inspect EMAIL, PHONE, PAN and AADHAAR placeholders.

Verify loan amounts remain readable.

PAIR LAB / Predict → run → inspect → explain / uv run --no-sync

Exercise / preserve the complete policy condition

```
Find one table, one exception and one date caveat.
```

Show the child text and its parent context.

Explain which change could create an incomplete answer.

PAIR LAB / Predict → run → inspect → explain / uv run --no-sync

Embeddings, Vector Storage, and Evaluation

Educative lesson 08

Concept → worked example → NexaAssist application

An embedding is a representation, not an answer

> A model maps text into numeric coordinates.

> Similarity helps rank candidates; it does not prove authority.

> Use the same compatible model and dimension schema.

Approximate search trades exhaustive work for speed

- Nearest-neighbor structures avoid scanning every vector.
- Index settings affect latency and candidate recall.
- Measure expected sources under the actual filters.

Embeddings are part of the schema

- Configured model and dimensions must match indexed vectors.
- The Atlas index points to the embedding field.
- A dimension mismatch is a compatibility failure.

Use the official MongoDB integration

```
return MongoDBAtlasVectorSearch(
    collection=_database(settings)[settings.mongodb_collection],
    embedding=embedding,
    index_name=settings.vector_index_name,
    text_key="text",
    embedding_key="embedding",
    relevance_score_fn="cosine",
    auto_create_index=False,
)
```

Collection, embedding and index_name are passed explicitly; index creation is manual.

The index definition is already documented

- Open app/database/indexes.md.
- Use the exact JSON matching configured dimensions.
- Check every metadata filter field in the live index.

Dates and permissions constrain evidence

```
Run: python -m training.session2.asof
```

Compare 2026-06-30 with 2026-07-01.

Expected: the July circular enters scope on its effective date.

PAIR LAB / Predict → run → inspect → explain / uv run --no-sync

Dense, hybrid and reranked retrieval

Dense search matches embedding similarity.

Hybrid adds lexical evidence; reranking changes order.

Evaluate each configuration on the same cases.

Hit rate and multi-reference recall are different

- Single expected source: hit or miss among top-k.
- Multiple expected clauses: fraction of required references found.
- MRR adds where the first expected item appears.

Recall and rank answer different questions

- Recall@k: fraction of required references found.
- MRR: reciprocal rank of the first expected reference.
- A fee question can require more than one clause.

Practical 3 / score a retrieval miss

```
Run: python -m training.session2.recall
```

Compare k=1, k=3 and k=5.

Explain the two-clause fee case and linked amendment.

PAIR LAB / Predict → run → inspect → explain / uv run --no-sync

Version the retrieval artifact as a compatible set

- Record parser/chunker settings and embedding identity.
- Build a candidate knowledge snapshot.
- Evaluate before changing the serving configuration.

The course retrieval CI gate is an example

- Source example: fail when single-reference hit rate < 0.90.
- NexaAssist has local recall gates with placeholder targets.
- A live Atlas test needs a real compatible candidate index.

Optional live / ingest then evaluate

- Needs approved Atlas access and model credentials.
- Run ingestion twice; then retrieval_eval --k 5.
- Record configuration, missing clauses and the gate result.

Treat re-indexing as a release

1. Build a separate compatible candidate data store.
2. Evaluate candidate retrieval and answer quality.
3. Switch the full configuration only after review.

Exit gate / evidence before eloquence

- Submit chunk experiment and checksum explanation.
- Report recall/MRR and categorize every missing reference.
- Live scores remain unmeasured until the live lab runs.

Next / control what generation may claim

Read app/chains/prompts/answer.py.

Bring one unsupported question and one scope ambiguity.

Keep the same retrieval cases for Session 3.